

Counter-Strike Polymarket Betting System — Technical Specification

Audience: an AI engineering agent implementing this system end-to-end. **Status:** v1 spec, targeting Counter-Strike 2 with a portable architecture for v2 (football, basketball, rugby).

1. Goals & Non-Goals

Goals

- Ingest, clean, and centralize all publicly available CS data (matches, players, teams, demos, social) from ~2020 onward.
- Engineer features that capture team, player, map, side, and contextual strength.
- Train probabilistic models that predict map and match outcomes, plus Polymarket-relevant derivative markets (map handicap, total maps, correct score).
- Compare model probabilities to Polymarket market prices, identify edge, and size bets with risk controls.
- Backtest rigorously with walk-forward validation; track live performance against the backtest distribution.
- Build the system modularly so v2 can plug in other sports without rewriting the core.

Non-Goals (v1)

- No execution bot for Polymarket. Bet sizing and edge are surfaced as recommendations; a human places trades.
 - No live in-play (mid-match) prediction. Pre-match only.
 - No deep NLP on streams for prediction features. Stream transcription is v1.5+, used only for qualitative context.
-

2. Architectural Principles

1. **Calibration > accuracy.** Optimize Brier score and log loss, not classification accuracy. A well-calibrated 51% model beats a miscalibrated 55% one.
 2. **Edge = model_prob – market_prob.** The model is half the system. Without market prices and an edge layer, this is academic.
 3. **No leakage, ever.** Every feature is computed with an explicit `as_of` timestamp. If a feature uses data not available before kickoff, it's a bug.
 4. **Rosters, not teams.** A CS "team" is its current five players. All team-level features must be computed against the roster that played that match.
 5. **Walk-forward validation only.** Random k-fold leaks future into past. Train on `[t0, t]`, validate on `(t, t+Δ]`, slide forward.
 6. **Sealed final hold-out.** Keep a final period (e.g. last 3 months of data) untouched during all development. Touch it once before deciding to deploy.
 7. **Idempotent pipelines.** Every ingestion and feature job must be re-runnable without duplicates or drift.
 8. **Reproducibility.** Every model artifact is tied to a Git SHA, a data snapshot ID, and a config hash.
-

3. System Architecture (High Level)

L1 INGESTION

HLTV scraper · Liquipedia API · Demo downloader · Polymarket
(v1.5: Twitter/X · Reddit · YouTube transcripts)



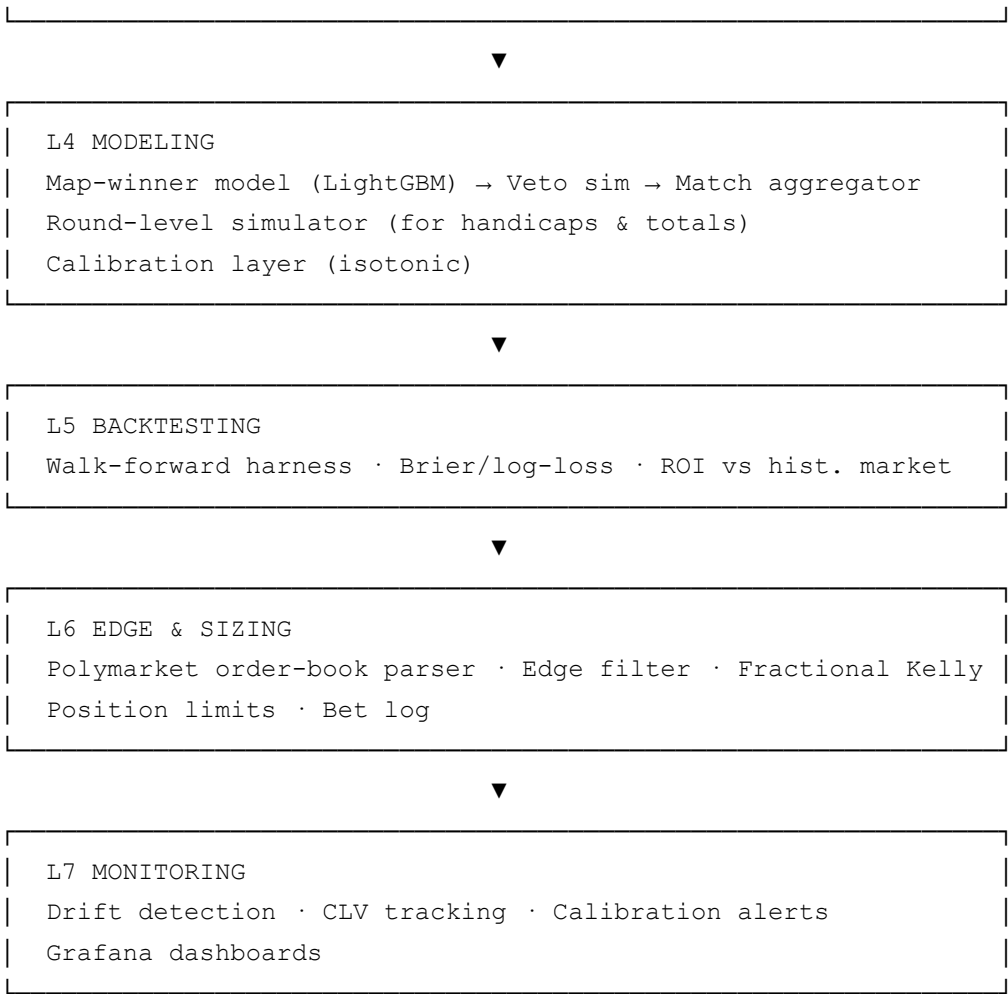
L2 STORAGE

Postgres (entities, matches, rosters)
ClickHouse / Parquet (tick & round-level demo data)
S3 (raw demos, raw HTML, raw JSON snapshots)
Redis (caches, rate limits, job locks)



L3 FEATURE STORE

Feast or homebrew. Point-in-time correct retrieval.
Same code path at train and predict time.



4. Layer 1 — Ingestion

4.1 Sources, in priority order

Source	What it gives you	Notes
HLTV	Matches, results, players, teams, rankings, stats, demo links	Hostile to scraping. Rate-limit aggressively (1 req / 3–5s), rotate residential IPs, respect robots, cache HTML. Use match ID as primary key.
Liquipedia	Tournaments, brackets, prize pools, rosters, transfers	MediaWiki API, friendly. Good source of truth for rosters and event metadata.
Demo files	Tick-by-tick player state, every kill, every	The richest data source by far. Download via

(.dem)	economy state, every utility throw	HLTV match page.
Tournament operator APIs	Live & historical match data	ESL, BLAST, PGL — sporadic, document per-TO.
Polymarket	Market prices, order books, resolved outcomes	CLOB API + GraphQL. Required for edge detection and backtesting against real prices.
(v1.5) Twitter/X	Player news, roster moves, injury hints	Use official lists: pro accounts, beat journalists.
(v1.5) Reddit r/GlobalOffensive	Community signal, narrative shifts	Lots of noise; deprioritize.
(v1.5) YouTube/Twitch VODs	Post-match interviews, analyst breakdowns	Transcribe with Whisper, extract entities.

4.2 Demo parsing

- Library: **awpy** (Python, well-maintained, returns Pandas DataFrames) as primary. Fallback to **demoinfocs-golang** (faster, lower-level) if **awpy** becomes a bottleneck.
- For each demo, extract and persist:
 - Match metadata: map, score, date, teams, players, tournament.
 - Per-round: round number, winner side, win condition (elim, bomb, defuse, time), economy state at freeze-time end, equipment value per side.
 - Per-kill: tick, attacker, victim, weapon, headshot flag, attacker position, victim position, was-trade, assister, flash assist.
 - Per-utility: thrower, type (flash/smoke/molly/HE), tick, landing position, effective duration.
 - Per-tick (optional, expensive): player positions at 1-Hz or 2-Hz. Only needed for advanced movement/positioning features. Skip in v1.
- Store raw demos in S3 with key `demos/{year}/{event_slug}/{match_id}/{map_name}.dem`. **Never delete; reparse on schema changes.**

4.3 Ingestion job design

- One job per source, scheduled via Airflow / Prefect / Dagster (pick one — recommend **Prefect** for ergonomics).
- All jobs are **incremental** and **idempotent**. Use `(source, source_id)` as a uniqueness constraint and upsert.
- Backfill flows run from a configurable start date; incremental flows run hourly/daily.
- Every fetched page/file is also persisted raw (HTML, JSON, .dem) in S3 with a content hash. Parsers run off the raw store, so reparsing is trivial when the schema evolves.

4.4 Polymarket ingestion

- Use the CLOB REST API and GraphQL subgraph.
 - For each CS-related market, snapshot the order book every N minutes (start at 5; tighten as needed).
 - Persist: market ID, slug, question, outcomes, resolved outcome (when known), per-snapshot best bid/ask/mid for each outcome.
 - For backtesting, the historical book is essential. Start ingesting **now**, even before the model is ready, because you can't reconstruct past prices.
-

5. Layer 2 — Storage Schema

5.1 Database choices

- **Postgres 16** for entities, relationships, and small/medium time series. Use partitioning on `matches` and `bets` by month.
- **ClickHouse** (or Parquet on S3 + DuckDB for development) for kills, rounds, utility events, ticks. Columnar, fast aggregations.
- **S3** for raw demos and raw HTML/JSON.
- **Redis** for caches, distributed locks, rate limiters.

5.2 Postgres schema (essentials)

```
-- Players
CREATE TABLE players (
  player_id      BIGSERIAL PRIMARY KEY,
  hltv_id        INT UNIQUE,
```

```

    liquipedia_slug TEXT UNIQUE,
    steam_id        TEXT UNIQUE,
    nickname        TEXT NOT NULL,
    real_name       TEXT,
    country         TEXT,
    birthdate       DATE,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Teams (organization-level)
CREATE TABLE teams (
    team_id          BIGSERIAL PRIMARY KEY,
    hltv_id          INT UNIQUE,
    liquipedia_slug TEXT UNIQUE,
    name             TEXT NOT NULL,
    country          TEXT,
    created_at       TIMESTAMPTZ DEFAULT NOW()
);

-- Roster history: which players were on which team between dates
-- This is the single most important table for avoiding "team strength" leakage.
CREATE TABLE roster_periods (
    roster_id        BIGSERIAL PRIMARY KEY,
    team_id           BIGINT REFERENCES teams,
    player_id        BIGINT REFERENCES players,
    role             TEXT,                -- rifler, awper, igl, support, entry, coach, sta
    start_date       DATE NOT NULL,
    end_date         DATE,                -- NULL = current
    is_stand_in      BOOLEAN DEFAULT FALSE,
    UNIQUE (team_id, player_id, start_date)
);

-- Events / tournaments
CREATE TABLE events (
    event_id         BIGSERIAL PRIMARY KEY,
    hltv_id          INT UNIQUE,
    name             TEXT,
    tier             TEXT,                -- S, A, B, C
    format           TEXT,                -- LAN, online
    prize_pool       NUMERIC,
    start_date       DATE,
    end_date         DATE,
    region          TEXT
);

-- Matches
CREATE TABLE matches (

```

```

match_id          BIGSERIAL PRIMARY KEY,
hlvtv_id          INT UNIQUE,
event_id          BIGINT REFERENCES events,
scheduled_at      TIMESTAMPTZ NOT NULL,
started_at        TIMESTAMPTZ,
ended_at          TIMESTAMPTZ,
team1_id          BIGINT REFERENCES teams,
team2_id          BIGINT REFERENCES teams,
best_of           INT,           -- 1, 3, 5
stage            TEXT,           -- groups, quarterfinal, etc.
is_lan            BOOLEAN,
status           TEXT,           -- scheduled, live, finished, cancelled, forfeit
winner_team_id    BIGINT REFERENCES teams,
team1_score       INT,
team2_score       INT
);

-- Map vetoes (in order)
CREATE TABLE veto_actions (
  match_id        BIGINT REFERENCES matches,
  order_idx       INT,
  team_id         BIGINT REFERENCES teams,
  action          TEXT,           -- ban, pick, decider
  map_name        TEXT,
  PRIMARY KEY (match_id, order_idx)
);

-- Maps played
CREATE TABLE map_results (
  map_result_id   BIGSERIAL PRIMARY KEY,
  match_id        BIGINT REFERENCES matches,
  map_index       INT,           -- 1..N within the match
  map_name        TEXT,           -- Mirage, Inferno, ...
  team1_score     INT,
  team2_score     INT,
  team1_ct_rounds INT,
  team1_t_rounds  INT,
  winner_team_id  BIGINT REFERENCES teams,
  duration_sec    INT,
  demo_s3_key     TEXT
);

-- Player participation per map (handles stand-ins)
CREATE TABLE map_lineups (
  map_result_id   BIGINT REFERENCES map_results,
  team_id         BIGINT REFERENCES teams,
  player_id       BIGINT REFERENCES players,

```

```

    PRIMARY KEY (map_result_id, player_id)
);

-- Per-player per-map performance (denormalized convenience table)
CREATE TABLE player_map_stats (
    map_result_id BIGINT REFERENCES map_results,
    player_id     BIGINT REFERENCES players,
    team_id      BIGINT REFERENCES teams,
    kills        INT,
    deaths       INT,
    assists      INT,
    adr          NUMERIC,
    kast         NUMERIC,
    rating_2_0   NUMERIC,
    hs_pct       NUMERIC,
    first_kills  INT,
    first_deaths INT,
    clutches_won INT,
    PRIMARY KEY (map_result_id, player_id)
);

-- Polymarket markets and snapshots
CREATE TABLE pm_markets (
    market_id    TEXT PRIMARY KEY,      -- Polymarket's ID
    slug         TEXT,
    question     TEXT,
    match_id     BIGINT REFERENCES matches, -- nullable until linked
    market_type  TEXT,                  -- match_winner, map_handicap, total_maps, correct_scor
    created_at   TIMESTAMPTZ,
    resolved_at  TIMESTAMPTZ,
    resolved_outcome TEXT
);

CREATE TABLE pm_book_snapshots (
    snapshot_id  BIGSERIAL PRIMARY KEY,
    market_id    TEXT REFERENCES pm_markets,
    taken_at     TIMESTAMPTZ NOT NULL,
    outcome      TEXT,
    best_bid     NUMERIC,
    best_ask     NUMERIC,
    mid          NUMERIC,
    depth_bid_1pct NUMERIC, -- liquidity within 1c of best bid
    depth_ask_1pct NUMERIC
);

CREATE INDEX ON pm_book_snapshots (market_id, taken_at DESC);

-- Bet log

```



```

CREATE TABLE bets (
  bet_id          BIGSERIAL PRIMARY KEY,
  match_id        BIGINT REFERENCES matches,
  market_id       TEXT REFERENCES pm_markets,
  outcome         TEXT,
  placed_at       TIMESTAMPTZ,
  model_prob      NUMERIC,
  market_prob     NUMERIC,
  edge            NUMERIC,
  kelly_fraction  NUMERIC,
  stake_usd       NUMERIC,
  price_in        NUMERIC,
  price_close     NUMERIC,
  clv             NUMERIC,
  resolved_outcome TEXT,
  pnl_usd         NUMERIC,
  model_version   TEXT,
  config_hash     TEXT
);

```

5.3 ClickHouse tables (sketch)

```

CREATE TABLE rounds (
  match_id UInt64, map_result_id UInt64, round_num UInt8,
  winner_side Enum('CT','T'), win_condition LowCardinality(String),
  ct_team_id UInt64, t_team_id UInt64,
  ct_eq_value UInt32, t_eq_value UInt32,
  ct_money UInt32, t_money UInt32,
  duration_sec UInt16,
  ts DateTime
) ENGINE = MergeTree ORDER BY (match_id, round_num);

```

```

CREATE TABLE kills (
  match_id UInt64, map_result_id UInt64, round_num UInt8, tick UInt32,
  attacker_id UInt64, victim_id UInt64, assister_id Nullable(UInt64),
  weapon LowCardinality(String), is_headshot UInt8, is_trade UInt8,
  attacker_pos_x Float32, attacker_pos_y Float32, attacker_pos_z Float32,
  victim_pos_x Float32, victim_pos_y Float32, victim_pos_z Float32,
  ts DateTime
) ENGINE = MergeTree ORDER BY (match_id, round_num, tick);

```

6. Layer 3 — Feature Engineering

6.1 Hard rule: point-in-time correctness

Every feature value is stamped with `(entity_id, as_of_timestamp, feature_value)`.

Training queries join features by `as_of < match.scheduled_at`. Prediction queries pass the kickoff time. The same code computes both.

6.2 Feature families

Team-map strength

- Per-map Glicko-2 rating (recommend Glicko-2 over Elo: handles inactivity and uncertainty).
- Per-map win rate over rolling 30 / 90 / 180 days.
- Side-specific CT and T round-win rates per map, last 90 days.
- Pistol-round and gun-round split.

Roster stability & context

- Days since last roster change.
- Number of stand-ins in this match (read from `map_lineups` vs `roster_periods`).
- Coach change in last 90 days (binary).

Player form

- Rating 2.0, ADR, KAST, impact, opening-duel success — rolling 30 / 90 / 180 days, weighted by opponent strength.
- AWPer flag and AWPer form (treat the team's primary AWPer's rating as its own feature; awping carries outsized variance).
- IGL flag.

Head-to-head

- H2H win rate, recency-weighted (exponential decay, half-life 180 days), same map only.
- H2H map-pool overlap.

Tournament context

- LAN vs online (binary; massive feature historically).
- Event tier.
- Stage (groups vs playoffs vs final).

- Best-of format.
- Days into tournament (fatigue / acclimation).
- Travel: regional vs cross-regional event.

Map pool & veto

- For each team, the probability they get to play each map *given the opponent's vetoes*. This is its own model — see §7.2.
- Forced-map indicator: how often a team plays this map as a decider vs a pick.

Meta & patch

- CS2 patch ID. After major patches (especially map reworks), reset or down-weight pre-patch data.

6.3 Implementation

- Use **Feast** with a Postgres offline store and Redis online store, or roll your own:
 - One materialization job per feature family, writing `(entity_id, as_of, value)` rows.
 - A `get_features(entity_ids, as_of)` function that returns the most recent value with `as_of_feature ≤ as_of_query`.
 - Cache aggressively but key caches on `(feature_name, entity_id, as_of_day)`.
-

7. Layer 4 — Modeling

7.1 Map-winner model

Target: `P(team1 wins map | features, map_name)`.

Algorithm: **LightGBM** binary classifier. Reasons: handles missing data, fast iteration, well-calibrated with isotonic post-processing, easy to interpret with SHAP.

Two variants, pick by validation:

1. Single model with `map_name` as a categorical feature.
2. One model per map (7 maps → 7 models). Less data per model, more specialization.

Training data: one row per (map_result, team1, team2) where `team1` is randomly assigned (each map appears twice with swapped roles) to remove ordering bias.

Features: difference and ratio of team-map features, plus context features (LAN, tier, BO, days rest, H2H, etc.). Avoid concatenating raw absolute values when a diff captures the signal.

Loss: binary log loss.

Calibration: fit isotonic regression on a held-out fold *after* training.

7.2 Veto / map-pick model

Target: $P(\text{map}_k \text{ chosen at veto step } s \mid \text{history, both teams' map-pool features})$.

Approach: multinomial classifier per veto step. Alternative: simulate the whole veto sequence with a small policy model trained on historical vetoes.

Output for downstream: for a BO3, a distribution over the 3-map set actually played.

7.3 Match aggregator

Given:

- Per-map win probabilities p_m from the map-winner model,
- A distribution over the played map set from the veto model,

simulate the BO3/BO5 via Monte Carlo (10k–100k runs):

```
def simulate_match(map_set_dist, p_win_per_map, best_of, n=50_000):
    counts = {"team1": 0, "team2": 0, "scores": Counter()}
    for _ in range(n):
        maps = sample(map_set_dist)
        wins1 = wins2 = 0
        target = (best_of // 2) + 1
        for m in maps:
            if random() < p_win_per_map[m]: wins1 += 1
            else: wins2 += 1
            if wins1 == target or wins2 == target: break
        winner = "team1" if wins1 > wins2 else "team2"
        counts[winner] += 1
        counts["scores"][(wins1, wins2)] += 1
    return normalize(counts)
```

This yields: match winner, exact score, map handicap, total maps — every common Polymarket market.

7.4 Round-level simulator (for handicaps / totals)

Build a separate gradient-boosted model on round-level data from demos:

```
P(CT wins round | map, side, ct_eq_value, t_eq_value, score_diff, round_num,
team_strengths)
```

Then simulate full maps (round by round, including the CS2 economy: loss bonus, equipment carryover, half-time side switch, MR12 overtime). This gives:

- Map total rounds.
- Round-handicap probabilities.
- Pistol round probabilities.

The two simulators (map-level and round-level) should agree at the map-winner margin within ~2 percentage points. If they don't, one is broken.

7.5 Calibration

After every training run:

- Plot a reliability diagram (predicted prob vs observed frequency, 10 bins).
- Compute Expected Calibration Error (ECE).
- Apply isotonic regression on a held-out window if ECE > 2%.
- Persist the calibrator with the model artifact.

7.6 Model artifacts

A model artifact is a directory containing:

- `model.lgb` — booster file.
- `calibrator.pkl` — isotonic regressor.
- `feature_spec.yaml` — exact feature names, dtypes, and `as_of` lookback rules.
- `metadata.json` — Git SHA, data snapshot ID, training window, validation metrics, config hash.

Stored in S3 under `models/{model_name}/{version}/`. Promotion to “prod” is an explicit pointer update, never an overwrite.

8. Layer 5 — Backtesting

8.1 Walk-forward harness

```
def walk_forward(matches_df, train_window_days=540, step_days=14):
    results = []
    t0 = matches_df.date.min() + timedelta(days=train_window_days)
    t = t0
    while t < matches_df.date.max():
        train = matches_df[(matches_df.date >= t - timedelta(days=train_window_days)
                             & (matches_df.date < t)]
        test = matches_df[(matches_df.date >= t)
                           & (matches_df.date < t + timedelta(days=step_days))]
        model = train_model(train)
        preds = model.predict(test)
        results.append(score(preds, test))
        t += timedelta(days=step_days)
    return aggregate(results)
```

8.2 Metrics to track

Per slice (overall, per map, per event tier, LAN vs online):

- **Log loss** (lower is better).
- **Brier score** (lower is better).
- **ECE** (lower is better).
- **AUC** (sanity check; do not optimize against this).
- **Sharpness** (mean absolute distance of preds from 0.5; rewards confident correct calls).
- **ROI vs historical Polymarket prices** at edge thresholds 2%, 4%, 6%, 8%.
- **Closing-line value (CLV)**: for each backtested bet, compare entry price to market close. Positive CLV is a leading indicator of long-run profit.

8.3 Sealed hold-out

Reserve the most recent 3 months of data. Never look at metrics on this slice during development. Touch once before deciding to deploy. If hold-out metrics diverge sharply from walk-forward, you've overfit your *process*.

9. Layer 6 — Edge Detection & Sizing

9.1 Implied probability from Polymarket

Polymarket binary outcomes trade in $[0, 1]$ cents. For an outcome priced at $\text{ask} = 0.62$:

- Implied probability = 0.62 (no fees baked in).
- Effective cost includes the bid-ask spread and (when withdrawing) USDC bridge friction.
- For a yes-bet:
 - $\text{effective_market_prob} = \text{ask}$
 - $\text{model_edge} = \text{model_prob} - \text{ask}$
 - Only consider markets where $\min(\text{depth_bid_1pct}, \text{depth_ask_1pct}) > \X (set X by bankroll, start at \$2k).

9.2 Edge filter

```
def is_actionable(model_prob, market_ask, min_edge=0.04, min_liquidity_usd=2000):  
    edge = model_prob - market_ask  
    return edge >= min_edge and depth_ok(market_ask, min_liquidity_usd)
```

Start with $\text{min_edge} = 0.04$ (4 percentage points). Tighten after the first 200 live bets if calibration holds.

9.3 Fractional Kelly

Full Kelly assumes a perfect model. Use $\frac{1}{4}$ **Kelly** in v1, raise to $\frac{1}{2}$ only after 6+ months of positive CLV and good calibration on live bets.

```
def kelly_fraction(p, price, fraction=0.25):  
    # p: model probability of YES; price: cost in [0,1] of YES share.  
    b = (1 - price) / price          # net odds  
    f_full = (p * b - (1 - p)) / b  
    return max(0.0, fraction * f_full)
```

$\text{Stake} = \text{bankroll} * \text{kelly_fraction}$, capped by:

- Per-bet cap: 2% of bankroll.
- Per-match cap: 4% of bankroll across all markets.
- Per-day cap: 8% of bankroll.

- Correlated markets (e.g., match winner + map handicap on same match) must share the per-match cap, not double-count.

9.4 Bet log discipline

Every recommendation is logged whether placed or not, with model prob, market prob, edge, recommended stake, and (if placed) actual stake. The log is the source of truth for whether the system has real edge.

10. Layer 7 — Monitoring & Ops

10.1 Live dashboards (Grafana)

- Calibration: rolling ECE on the last 100 / 500 bets.
- CLV distribution: histogram + rolling mean.
- ROI: cumulative PnL, drawdown, daily/weekly returns.
- Coverage: how many CS markets the system priced today vs total available.
- Latency: model inference, feature retrieval, Polymarket snapshot.

10.2 Drift detection

- Feature drift: PSI (Population Stability Index) on top-20 features, week-over-week. Alert above 0.2.
- Prediction drift: KS test on the distribution of model probs week-over-week.
- Performance drift: alert if rolling 100-bet log loss exceeds backtest 95th percentile.

10.3 Kill switches

Auto-pause new recommendations if:

- Rolling 50-bet CLV goes negative for two consecutive weeks, or
 - Calibration ECE on rolling 200 bets exceeds 5%, or
 - A feature pipeline is more than 6 hours stale.
-

11. v2 Portability (Football, Basketball, Rugby)

11.1 What ports cleanly

- Ingestion framework (sources change; the scheduler and idempotency layer don't).
- Storage skeleton (teams , players , matches , events , roster_periods , bets , pm_markets , pm_book_snapshots).
- Feature store interface.
- Training and backtesting harness.
- Calibration layer.
- Edge detection and Kelly sizing.
- Monitoring stack.

11.2 What doesn't port

- Demo parsing (CS-specific).
- Map / veto logic (CS-specific; basketball has no maps but has home/away; football has venue & weather).
- Round simulator.
- Feature definitions (sport-specific).
- Most external sources.

11.3 Plugin interface

Define `Sport` as an ABC:

```
class Sport(ABC):
    name: str
    @abstractmethod
    def ingest(self, since: date) -> None: ...
    @abstractmethod
    def build_features(self, as_of: datetime, entities: list[int]) -> pd.DataFrame
    @abstractmethod
    def train(self, train_window: tuple[date, date]) -> ModelArtifact: ...
    @abstractmethod
    def predict(self, match: Match, artifact: ModelArtifact) -> dict[str, float]:
    @abstractmethod
    def simulate_markets(self, predictions: dict, match: Match) -> dict[str, floa
```

Each sport ships as a package implementing this interface. The orchestrator, edge layer,

sizing layer, and monitoring know nothing about which sport they're running.

12. Tech Stack Summary

Concern	Choice	Rationale
Language	Python 3.12	Ecosystem (awpy, lightgbm, pandas).
Pipeline orchestration	Prefect 3	Pythonic, good local dev story.
Demo parsing	awpy (primary), demoinfocs-golang (fallback)	Best-maintained CS demo libraries.
ML	LightGBM, scikit-learn, optuna	Fast, calibratable, hyperparameter tuning.
RDBMS	Postgres 16	Solid, partitioning-friendly.
Columnar	ClickHouse (or DuckDB + Parquet in dev)	Fast aggregations on tick data.
Object storage	S3 (or MinIO locally)	Raw demos & HTML.
Cache / locks	Redis	Rate limits, job locks.
Feature store	Feast	Or homebrew if Feast feels heavy.
Monitoring	Prometheus + Grafana	Standard.
Experiment tracking	MLflow	Models, metrics, params.
Infra	Docker Compose (dev), Kubernetes (prod, when justified)	Start simple.
CI	GitHub Actions	Test + lint + package + deploy.

13. Repository Layout

```
cs-bet/
├─ README.md
├─ pyproject.toml
├─ docker-compose.yml
├─ infra/
│   └─ terraform/
├─ config/
│   ├── base.yaml
│   ├── dev.yaml
│   └─ prod.yaml
├─ sports/
│   ├── base/
│   │   └─ interface.py
│   └─ cs/
│       ├── ingest/
│       │   ├── hltv.py
│       │   ├── liquipedia.py
│       │   ├── demos.py
│       │   └─ polymarket.py
│       ├── parse/
│       │   ├── demo_parser.py
│       │   └─ round_extractor.py
│       ├── features/
│       │   ├── team_map.py
│       │   ├── player_form.py
│       │   ├── h2h.py
│       │   └─ context.py
│       ├── models/
│       │   ├── map_winner.py
│       │   ├── veto.py
│       │   ├── round_sim.py
│       │   └─ aggregator.py
│       └─ markets/
│           └─ pm_mapper.py
├─ core/
│   ├── feature_store/
│   ├── backtest/
│   ├── edge/
│   ├── sizing/
│   ├── monitoring/
│   └─ storage/
├─ migrations/          # Alembic
├─ tests/
└─ notebooks/           # exploration only, never imported
```

14. Implementation Roadmap

Phase 0 — Foundations (Week 1–2)

- Repo scaffolding, CI, Docker Compose with Postgres, Redis, MinIO, ClickHouse.
- Postgres migrations for §5.2 schema.
- Start Polymarket ingestion **now**, even before models exist.

Phase 1 — Data layer (Week 3–6)

- HLTV scraper + Liquipedia client.
- Demo downloader and `awpy` parser.
- Backfill 2022–present matches, results, demos.
- Validate: every match has a roster, every map has a demo or an explicit “demo unavailable” flag.

Phase 2 — Feature store (Week 7–9)

- Implement team-map ratings (Glicko-2), player form, H2H, context features.
- Point-in-time retrieval API.
- Unit tests with synthetic leak attempts that must fail.

Phase 3 — First model (Week 10–12)

- Map-winner LightGBM, single model with map as feature.
- Walk-forward harness, calibration, baseline metrics.
- Compare to a naïve baseline (HLTV ranking diff). Must beat it on log loss.

Phase 4 — Veto + aggregator + round sim (Week 13–16)

- Veto model, Monte Carlo aggregator, round-level simulator.
- Derive map handicap, total maps, correct score probabilities.

Phase 5 — Edge + paper trading (Week 17–20)

- Polymarket order book ingestion fully wired.
- Edge filter, Kelly sizing, full bet log.

- **Paper trade for 60 days.** Track CLV and calibration.

Phase 6 — Go-live (Week 21+)

- Touch the sealed hold-out. If metrics hold, deploy with small bankroll.
- Gradually raise bankroll only as CLV stays positive.

Phase 7 — v1.5 enhancements

- Stream transcription, social signal, news event detection.
- Live in-play model (separate project).

Phase 8 — v2 port

- Refactor core to enforce the `Sport` interface.
- Pilot one new sport (recommend basketball: clean data, deep markets).

15. Risks & Mitigations

Risk	Mitigation
Polymarket CS liquidity is thin outside majors	Track depth per market; only size into liquid books. Focus on majors and tier-1 events.
Sharp markets eat naïve edge	Hunt in correlated/exotic markets and react fast to news. Measure CLV, not just PnL.
Roster changes invalidate team features overnight	Re-rate immediately on roster change; down-weight to “low confidence” for first N matches with new lineup.
Online vs LAN regime shift	Carry <code>is_lan</code> as a hard feature, train enough LAN data, monitor split metrics.
Patch/meta shifts (map reworks, weapon balance)	Versioned patch IDs as feature. Inspect SHAP weekly for unexplained shifts.
Overfitting via repeated backtest peeking	Sealed hold-out, frozen pre-registered metric thresholds.
HLTV blocks scraper	Polite rate-limits, rotating IPs, raw HTML cache so reparses don't re-fetch, fallback to Liquipedia + demos.

Demo unavailable for some matches	Flag and exclude from demo-dependent features; team-level features still computable from HLTV stats.
Polymarket terms/region restrictions	Out of scope for engineering; flagged to user.

16. Definition of Done (for the AI agent)

Each module is “done” when:

1. It has unit tests covering happy path and at least two failure modes.
2. It has an integration test exercising the real schema and (where applicable) a recorded fixture from a live source.
3. It has a leak test: a deliberately-future-leaked input must produce a test failure.
4. It logs structured events to a central logger with `run_id`, `git_sha`, `config_hash`.
5. It exposes a CLI entrypoint (`python -m sports.cs.ingest.hltv backfill --since 2022-01-01`).
6. It has a one-page docstring at the top of the main module explaining purpose, inputs, outputs, and known limitations.

The **system** is done (v1) when:

- 60 days of paper-trading shows positive CLV with rolling ECE < 3%.
 - Sealed hold-out metrics are within 1 standard deviation of walk-forward metrics.
 - Dashboards, kill switches, and bet log are live and tested end-to-end.
-

17. Open Questions for the User

1. Bankroll size and risk tolerance — sets per-bet caps and edge thresholds.
2. Geography / Polymarket account access — affects execution path.
3. Preferred cloud provider (AWS / GCP / bare metal) — sets infra choices.
4. Is stream transcription a must-have for v1, or acceptable in v1.5?
5. v2 sport priority order — basketball, football (soccer), American football, rugby?

6. Compute budget — caps demo-parsing parallelism and model search.